

Name: A Nandini

Hall Ticket No: 2303A51512

Batch: 26

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING																																					
Program Name: B. Tech		Assignment Type: Lab	Academic Year: 2025-2026																																				
Course Coordinator Name		Dr. Rishabh Mittal																																					
Instructor(s)Name		<table border="1"> <tr><td>Mr. S Naresh Kumar</td><td></td></tr> <tr><td>Ms. B. Swathi</td><td></td></tr> <tr><td>Dr. Sasanko Shekhar Gantayat</td><td></td></tr> <tr><td>Mr. Md Sallauddin</td><td></td></tr> <tr><td>Dr. Mathivanan</td><td></td></tr> <tr><td>Mr. Y Srikanth</td><td></td></tr> <tr><td>Ms. N Shilpa</td><td></td></tr> <tr><td>Dr. Rishabh Mittal (Coordinator)</td><td></td></tr> <tr><td>Dr. R. Prashant Kumar</td><td></td></tr> <tr><td>Mr. Ankushavali MD</td><td></td></tr> <tr><td>Mr. B Viswanath</td><td></td></tr> <tr><td>Ms. Sujitha Reddy</td><td></td></tr> <tr><td>Ms. A. Anitha</td><td></td></tr> <tr><td>Ms. M.Madhuri</td><td></td></tr> <tr><td>Ms. Katherashala Swetha</td><td></td></tr> <tr><td>Ms. Velpula sumalatha</td><td></td></tr> <tr><td>Mr. Bingi Raju</td><td></td></tr> <tr><td>Mr. G. Kranthi</td><td></td></tr> </table>		Mr. S Naresh Kumar		Ms. B. Swathi		Dr. Sasanko Shekhar Gantayat		Mr. Md Sallauddin		Dr. Mathivanan		Mr. Y Srikanth		Ms. N Shilpa		Dr. Rishabh Mittal (Coordinator)		Dr. R. Prashant Kumar		Mr. Ankushavali MD		Mr. B Viswanath		Ms. Sujitha Reddy		Ms. A. Anitha		Ms. M.Madhuri		Ms. Katherashala Swetha		Ms. Velpula sumalatha		Mr. Bingi Raju		Mr. G. Kranthi	
Mr. S Naresh Kumar																																							
Ms. B. Swathi																																							
Dr. Sasanko Shekhar Gantayat																																							
Mr. Md Sallauddin																																							
Dr. Mathivanan																																							
Mr. Y Srikanth																																							
Ms. N Shilpa																																							
Dr. Rishabh Mittal (Coordinator)																																							
Dr. R. Prashant Kumar																																							
Mr. Ankushavali MD																																							
Mr. B Viswanath																																							
Ms. Sujitha Reddy																																							
Ms. A. Anitha																																							
Ms. M.Madhuri																																							
Ms. Katherashala Swetha																																							
Ms. Velpula sumalatha																																							
Mr. Bingi Raju																																							
Mr. G. Kranthi																																							
Course Code	23CS002PC304	Course Title	AI Assisted Coding																																				
Year/Sem	III/I	Regulation	R23																																				
Date and Day of Assignment	Week 5 - Thursday	Time(s)	23CSBTB01 To 23CSBTB52																																				
Duration	2 Hours	Applicable to Batches	All Batches																																				
AssignmentNumber: 10.4 (Present assignment number)/24(Total number of assignments)																																							
Q.No.	Question		ExpectedTime to complete																																				
1	Lab 9 – Code Review and Quality: Using AI to Improve Code Quality and Readability		Week 5																																				

	Lab Objectives • Use AI for automated code review and quality enhancement. • Identify and fix syntax, logical, performance, and security issues in Python code. • Improve readability and maintainability through structured refactoring and comments. • Apply prompt engineering for targeted improvements. • Evaluate AI-generated suggestions against PEP 8 standards and software engineering best practices	
	Task 1: AI-Assisted Syntax and Code Quality Review Scenario You join a development team and are asked to review a junior developer's Python script that fails to run correctly due to basic coding mistakes. Before deployment, the code must be corrected and standardized. Task Description You are given a Python script containing: • Syntax errors • Indentation issues • Incorrect variable names • Faulty function calls Use an AI tool (GitHub Copilot / Cursor AI) to: • Identify all syntactic and structural errors • Correct them systematically • Generate an explanation of each fix made Expected Outcome • Fully corrected and executable Python code • AI-generated explanation describing: ◦ Syntax fixes ◦ Naming corrections ◦ Structural improvements • Clean, readable version of the script	

	<pre>1 def great_num(int num): 2 if(num > 100): 3 return "The number is greater t 4 else 5 retur "not great 6 print(great_num(150))"" 7 # The code has several syntax errors and logical issues generate the correct code 8 # Corrected code: 9 def great_num(num): 10 if num > 100: 11 return "The number is greater than 100" 12 else: 13 return "The number is not greater than 100" 14 print(great_num(150)) 15 # Explanation: 16 # The corrected code defines a function `great_num` that takes an integer `num` as 17 # and checks if it is greater than 100. It returns a string message based on the co</pre>	
	Task 2: Performance-Oriented Code Review Scenario A data processing function works correctly but is inefficient and slows down the system when large datasets are used. Task Description You are provided with a function that identifies duplicate values in a list using inefficient nested loops. Using AI-assisted code review: • Analyze the logic for performance bottlenecks • Refactor the code for better time complexity • Preserve the correctness of the output Ask the AI to explain: • Why the original approach was inefficient • How the optimized version improves performance Expected Outcome • Optimized duplicate-detection logic (e.g., using sets or hash-based structures) • Improved time complexity • AI explanation of performance improvement • Clean, readable implementation	

```

1  def print_duplicates(numbers):
2      duplicates = []
3      for i in range(len(numbers)):
4          for j in range(i + 1, len(numbers)):
5              if numbers[i] == numbers[j] and numbers[i] not in duplicates:
6                  duplicates.append(numbers[i])
7      return duplicates
8  numbers = [1, 2, 3, 4, 5, 2, 3, 6]
9  print("Duplicate numbers:", print_duplicates(numbers))
10 #Make this code more efficient by using a set to track seen numbers and duplicates.
11 def print_duplicates(numbers):
12     seen = set()
13     duplicates = set()
14     for number in numbers:
15         if number in seen:
16             duplicates.add(number)
17         else:
18             seen.add(number)
19     return list(duplicates)
20 numbers = [1, 2, 3, 4, 5, 2, 3, 6]
21 print("Duplicate numbers:", print_duplicates(numbers))
22 #Explanation:
23 #The original code uses a nested loop to compare each number with every other number,
24 #which results in a time complexity of O(n^2). The improved code uses a set to track seen
25 #and duplicates, which allows it to check for duplicates in O(1) time. |

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

python\10_4.py'
Duplicate numbers: [2, 3]
Duplicate numbers: [2, 3]

```

Task 3: Readability and Maintainability Refactoring

Scenario

A working script exists in a project, but it is difficult to understand due to poor naming, formatting, and structure. The team wants it rewritten for long-term maintainability.

Task Description

You are given a poorly structured Python function with:

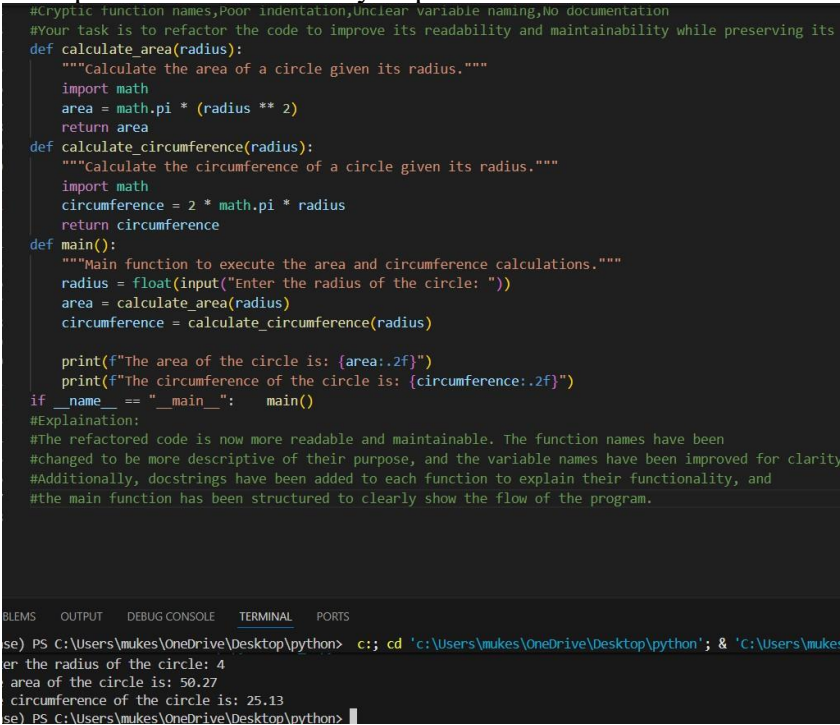
- Cryptic function names
- Poor indentation
- Unclear variable naming
- No documentation

Use AI-assisted review to:

- Refactor the code for clarity
- Apply PEP 8 formatting standards
- Improve naming conventions
- Add meaningful documentation

Expected Outcome

- Clean, well-structured code

	• Descriptive function and variable names • Proper indentation and formatting • Docstrings explaining the function purpose • AI explanation of readability improvements  <pre>#Cryptic function names,Poor indentation,Unclear variable naming,No documentation #Your task is to refactor the code to improve its readability and maintainability while preserving its def calculate_area(radius): """Calculate the area of a circle given its radius.""" import math area = math.pi * (radius ** 2) return area def calculate_circumference(radius): """Calculate the circumference of a circle given its radius.""" import math circumference = 2 * math.pi * radius return circumference def main(): """Main function to execute the area and circumference calculations.""" radius = float(input("Enter the radius of the circle: ")) area = calculate_area(radius) circumference = calculate_circumference(radius) print(f"The area of the circle is: {area:.2f}") print(f"The circumference of the circle is: {circumference:.2f}") if __name__ == "__main__": main() #Explanation: #The refactored code is now more readable and maintainable. The function names have been #changed to be more descriptive of their purpose, and the variable names have been improved for clarity #Additionally, docstrings have been added to each function to explain their functionality, and #the main function has been structured to clearly show the flow of the program.</pre> <pre>se) PS C:\Users\mukes\OneDrive\Desktop\python> c:: cd 'c:\Users\mukes\OneDrive\Desktop\python'; & 'c:\Users\muke er the radius of the circle: 4 The area of the circle is: 50.27 The circumference of the circle is: 25.13 se) PS C:\Users\mukes\OneDrive\Desktop\python></pre>	
	Task 4: Secure Coding and Reliability Review Scenario A backend function retrieves user data from a database but has security vulnerabilities and poor error handling, making it unsafe for production deployment. Task Description You are given a Python script that: • Uses unsafe SQL query construction • Has no input validation • Lacks exception handling Use AI tools to: • Identify security vulnerabilities • Refactor the code using safe coding practices • Add proper exception handling • Improve robustness and reliability Expected Outcome	

	• Secure SQL queries using parameterized statements • Input validation logic • Try-except blocks for runtime safety • AI-generated explanation of security improvements • Production-ready code structure <pre>#A backend function retrieves user data from a database but has security # vulnerabilities and poor error handling, making it unsafe for production deployment #Refactor the code using safe coding practices import sqlite3 def get_user_data(user_id): try: # Connect to the database securely using parameterized queries conn = sqlite3.connect('user_data.db') cursor = conn.cursor() # Use parameterized query to prevent SQL injection cursor.execute("SELECT * FROM users WHERE id = ?", (user_id,)) user_data = cursor.fetchone() if user_data: return user_data else: return "User not found." except sqlite3.Error as e: # Log the error securely without exposing sensitive information print(f"Database error: {e}") return "An error occurred while retrieving user data." finally: # Ensure the database connection is closed properly if conn: conn.close() # Example usage user_id = 1 print(get_user_data(user_id))</pre> TERMINAL <pre>PS C:\Users\mukes\OneDrive\Desktop\python> .\python3.13.exe 'c:\Users\mukes\.vscode\extensions\ms-python.debugpy-2025.18.0-w Database error: no such table: users An error occurred while retrieving user data. PS C:\Users\mukes\OneDrive\Desktop\python></pre>	
	Task 5: AI-Based Automated Code Review Report Scenario Your team uses AI tools to perform automated preliminary code reviews before human review, to improve code quality and consistency across projects. Task Description You are provided with a poorly written Python script. Using AI-assisted review: • Generate a structured code review report that evaluates: ○ Code readability ○ Naming conventions ○ Formatting and style consistency	

- Error handling
- Documentation quality
- Maintainability

The task is not just to fix the code, but to **analyze and report on quality issues**.

Expected Outcome

- AI-generated review report including:
 - Identified quality issues
 - Risk areas
 - Code smell detection
 - Improvement suggestions
- Optional improved version of the code
- Demonstration of AI as a **code reviewer**, not just a code generator

Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots

```
> Users > mukes > OneDrive > Desktop > python > 10.4.py > ...
1 #You are provided with a poorly written Python script.,Generate a structured code review report that evaluates:
2 #Code readability,Naming conventions,Formatting and style consistency,Error handling
3 #Documentation quality,Maintainability generate a code review report for the following script:``python
4 def calc(a,b):
5     return a+b
6 def main():
7     x=10
8     y=20
9     result=calc(x,y)
10    print("The result is:",result)
11 if __name__=="__main__":    main()
12 #Explanation:
13 #Code Review Report
14 #1. Code Readability:
15 # - The code is simple and straightforward, making it easy to understand.
16 #2. Naming Conventions:
17 # - The function name 'calc' is not descriptive. It would be better to use a more specific name like 'add_numbers'.
18 # - Variable names 'a', 'b', 'x', and 'y' are not descriptive. Consider using names like 'num1', 'num2', 'first numb
```